User Guide

Ask a question in plain English — get validated, read-only SQL, a live result table, and an auto-picked chart.

Documentation revision: 2026-10-15. Describes the application as of commit `c0f4271`. Features added after that date will not appear here.

Generated 2026-10-15 by `scripts/build_user_guide_pdf.py` from `USER_GUIDE.md` — the single source of truth for this document. Re-run the script after editing the markdown; do not hand-edit this PDF.

Contents

1. Starting the application 5
2. Connecting and configuring a database 6
3. Working with multiple databases 7
4. Loading and indexing the schema 8
5. Asking a natural-language question 9
6. How schema retrieval works (in plain terms) 10
7. Reviewing the generated SQL 11
8. SQL safety and validation 12
9. Reviewing query cost 13
10. Confirming execution 14
11. Viewing results 15
12. Charts and AI insights 16
13. Editing SQL yourself 17
14. Retry and self-correction behavior 18
15. Follow-up questions 19
16. Multi-source knowledge: documents, policies, and web search 20
17. Voice input and spoken answers 22
18. Generating images and video 23
19. Searching your media library 24
20. Conversation history 25
21. Understanding error and status messages 26
22. Security — what this app does and doesn't protect against 27
23. Troubleshooting 28
24. Architecture overview 29
25. The question-answering pipeline, step by step 31
26. Schema-aware retrieval, technically 33
27. SQL safety architecture 34
28. Self-correction and the adaptive retry budget 35

29. Agentic query planning and plan-conformance review 36
30. Multi-database auto-routing, technically 37
31. Multi-source orchestration architecture 38
32. Configuration and deployment 40
33. Security model summary 41
34. Further technical reading 42

A complete walkthrough of the Text-to-SQL Dashboard, covering both what it is like to use it and how it actually works. This guide has two parts:

- **Part 1 — Using the application (§1–§23)**: a plain-language walkthrough of what you see, what each button does, and what the messages mean. No prior technical knowledge is assumed.
- **Part 2 — Technical overview (§24–§34)**: how the application is built — the architecture, the design decisions behind the behavior described in Part 1, and the mechanisms underneath it — written for a reader who wants real understanding, not just operating instructions (a developer, an administrator troubleshooting something non-obvious, or a technical reviewer evaluating the project). Part 2 stands on its own, but is a condensed, self-contained summary, not a replacement for `docs/ARCHITECTURE.md` and `SECURITY.md`, the full, most detailed technical references, which Part 2 links to throughout for anyone who wants to go deeper.

This guide describes the React dashboard (`frontend/`, served at `http://localhost:8000/` once built — see `README.md`'s "Getting Started"); if your organization exposes the REST API instead, the underlying behavior described in both parts is identical, but there's no chat window — see that document for the programmatic equivalent. **Note:** this guide was originally written against an earlier Streamlit interface; Part 1's underlying behavior and Part 2's architecture are current, but some section-by-section UI descriptions (exact button labels, panel names) may not perfectly match the current dashboard's layout everywhere — the dashboard's own tooltips and labels are the source of truth if the two ever disagree.

Quick start: type a question in the chat box → review the SQL the AI proposes (nothing has run yet) → click **"Confirm and Run"** → read the results. Part 1 explains what happens at each of those steps and what to do when something doesn't go as expected; Part 2 explains why it behaves that way.

Several things described in Part 1 are **optional, admin-configured features** — skip straight past any section whose intro says it doesn't apply to your setup; everything else in this guide works identically either way:

- Asking about more than one database at once (§3).
- Asking about uploaded documents, company policies, or the live web in addition to your database (§16).
- Talking to the app out loud instead of typing (§17) — this one is **on** by default, so you'll usually see it unless your administrator turned it off.
- Asking the app to create a new image or video (§18) — off by default.
- Asking the app to find an existing photo or video in a local media library (§19) — off by default.

1. Starting the application

Once installed and configured (see `README.md`'s Getting Started section), build and start the app with:

```
cd frontend && npm install && npm run build && cd ..
uvicorn api.main:app --host 0.0.0.0 --port 8000
```

Or, if it's running via Docker (see `docs/DEPLOYMENT.md`), just open `http://localhost:8000` in a browser — everything described below is identical either way; only how the app is *started* differs.

Open `http://localhost:8000` in your browser. A persistent left-hand sidebar lists your past conversations (search box and "New chat" button at the top); the header above the main chat area shows the product name, a sidebar-collapse toggle, and — top right — your account menu (an avatar), which opens **Settings** (appearance, database connection status, discovered tables, and toggles like "Generate AI insight" and "Voice mode" — see §4/§12) as well as Theme and Sign out. Before you've asked anything, the main panel shows a heading ("Ask your data anything"), a short explanation, and a few example questions you can click to try immediately.

If the database can't be reached, the chat UI still loads — there is no full-screen blocking error. Open **Settings** (§2) to see the specific connection error per database, or run `python scripts/test_db_connection.py` for a detailed diagnostic. Asking a question against a database that can't be reached fails with an error at that point rather than earlier.

2. Connecting and configuring a database

There's no in-app login or connection form — the database is configured once, in `.env`, before starting the app (see `docs/CONFIGURATION.md`). Once running, open **Settings** (account menu, top right → "Settings") — its **"Connection status"** section shows what's currently connected: database type, database name, a partially-masked username (e.g. `j***n`), and schema (if restricted to one). A **"Test Connection"** button re-checks the connection at any time and reports success (with the database version) or the specific error.

If the connected account happens to have write access (INSERT/UPDATE/ DELETE), a warning banner appears here too. This app never issues writes itself, but that warning is worth acting on — see `SECURITY.md` for why a genuinely read-only account matters.

3. Working with multiple databases

(Only relevant if your administrator configured more than one database — skip to §4 otherwise.)

If your `.env` lists more than one database connection, Settings' **"Connection status"** section (§2) shows one card per database — each with its own status (reachable/not reachable), type, name, user, and schema, so you can tell at a glance which ones are reachable.

There is no manual "pick a database" dropdown. You just ask your question in plain English, and the app automatically figures out which configured database it's actually about — by comparing your question against each database's own table structure and picking the best match, before generating any SQL. You can see which one it picked in the answer's collapsed **"Query information"** panel (a **"Database"** field) — technical metadata that stays out of the way unless you expand it.

The **"Refresh Schema"** button (§4) refreshes *every* configured database in one click — the success message reports the total table count across all of them. The **"Discovered tables"** section (grouped by database when more than one is configured) lets you confirm the app sees the right tables in the right place.

If a question gets routed to the wrong database, it's almost always because the question is genuinely ambiguous between two databases that happen to share similar-sounding table/column names — try mentioning a distinctive term from the database you actually mean (a specific table name, or a business term unique to that data) rather than a generic phrase.

4. Loading and indexing the schema

Before the app can answer questions, it needs to know your database's shape. This happens automatically once per app run, and is also available on demand via Settings' **"Refresh Schema"** button (§2) — use it after you've added, renamed, or removed tables/columns in the database. It re-reads the schema and rebuilds the internal search index; a success message shows how many tables were found. This is safe to click any time — if nothing actually changed, it's a fast no-op.

Settings' **"Discovered tables"** section (collapsed by default) lists every table the app currently knows about — useful for confirming the app can see what you expect it to.

5. Asking a natural-language question

Type your question into the chat box at the bottom — *"Ask a question about your data..."* — and press Enter. Your question appears in the chat immediately, followed by the assistant's response once processing finishes (a spinner reads *"Retrieving schema, generating SQL, self-correcting if needed..."* while this happens).

If you ask the exact same question again in the same session (with the same prior conversation context and insight setting), you'll see *"(served from this session's cache — no LLM call made)"* — an instant answer with no repeat processing.

6. How schema retrieval works (in plain terms)

Your database might have dozens or hundreds of tables. Rather than showing the AI every single one (which would be slow and error-prone), the app finds and shows it only the tables most likely relevant to your question — plus any additional tables structurally required to connect them (e.g. a linking table between two dimensions). You can see exactly which tables were used for your question in the **"Retrieved schema context"** expander, including each table's relevance score and its structure.

7. Reviewing the generated SQL

After processing, the SQL query the AI generated appears under **"Generated SQL"** in an editable text box, with the note: *"Edit if needed — it will be re-validated and re-run when you click Confirm and Run."* **Nothing has been run against your database yet at this point** — this is a proposal for you to review, not a completed action.

You're free to edit the SQL directly in the box before running it — fix a typo, add a filter, change a column — your edit is what actually runs, and it's checked for safety fresh, exactly like AI-generated SQL (see the next section).

8. SQL safety and validation

Every query — whether generated by the AI or edited by hand — is checked before it's allowed to run. Only a single, read-only `SELECT`-style query is ever permitted; anything else (an attempt to modify data, run more than one statement, or call a small number of known-risky database functions) is rejected outright, with no retry. This check happens **every time**, including on your own hand-edits — there's no way to bypass it from inside the app. See `SECURITY.md` for the full technical detail, and its "What is explicitly not guaranteed" section for this protection's honest limits.

9. Reviewing query cost

Before a query actually runs, the app estimates roughly how much work it will involve (without running it). If a query looks like it will scan a lot of data, you'll see a caption: *"This query scans a large amount of data and may take a moment to run."* — informational, the query still runs. If a query looks extremely expensive, it isn't run at all; the AI is asked to try a narrower approach instead (this shows up as a step in the retry timeline — see below — rather than a message to you directly).

10. Confirming execution

Click **"Confirm and Run"** to actually execute the SQL currently shown in the box (including any edits you made). This is the one moment anything runs against your real database for something you'll actually see. If the SQL fails the safety check at this point (e.g. you edited it into something unsafe), you'll see *"Rejected: ..."* and nothing runs. If it runs but the database itself returns an error, you'll see *"Execution failed: ..."*.

11. Viewing results

A successful run shows a results table headed **"Results (N rows)"**. By default, purely technical columns (internal ID/key columns) are hidden and column names are expanded into readable labels (e.g. `CustName` becomes "Customer Name") — check **"Show technical columns"** to see the raw, unmodified column names and every column instead.

If nothing but ID-type columns came back, you'll see a note: *"Only identifier columns were returned."*

12. Charts and AI insights

If the result shape supports it (at least one numeric and one descriptive column), a **"Show chart"** checkbox becomes available — a line chart for results that look like a trend over time, otherwise a bar chart (capped to the top 30 values for readability). If the result shape doesn't fit either, the checkbox is disabled with the note *"No suitable chart for this result shape"* — the app never forces a misleading chart.

If enabled (Settings toggle, on by default: **"Generate AI insight"** — see §2/§4), a short plain-English sentence about the result appears above the table, labeled **"AI insight."** This sentence is checked against the actual result numbers before being shown — if it doesn't hold up, it's silently skipped rather than shown as an unverified guess. Turn the Settings toggle off if you'd rather not see this at all.

13. Editing SQL yourself

As covered in §7/§10: the SQL box is always editable, and clicking **"Confirm and Run"** always uses whatever text is currently in the box — not necessarily what the AI originally proposed. Every edit goes through the same safety check as AI-generated SQL, with no exceptions.

14. Retry and self-correction behavior

When the AI's first attempt at SQL doesn't work — a syntax mistake, a reference to a column that doesn't exist, an overly expensive query — the app doesn't just give up. It automatically tries again (up to a configured limit, widened automatically for a question that looks like it needs more attempts — see below), using what went wrong as guidance for the next attempt. You can see this entire process in the **"Retry timeline"** expander: one line per attempt, showing what happened (succeeded, timed out, too costly, etc.), the SQL that was tried, and the error if any. This is a summary of outcomes only — it does not show the AI's internal reasoning, only what it tried and what happened.

A summary line also tells you at a glance: *"Generated SQL after 2 retries."* (or *"Generated SQL."* if it worked on the first try).

Some problems are never retried — a safety rejection or a genuine timeout ends the attempt immediately rather than trying again, since retrying wouldn't help.

For a harder question — one that asks for something like "top 3 products per region" or "year-over-year growth" — you may also see a **"Query plan"** expander above the SQL box. Behind the scenes, the app recognized the question as the kind that benefits from planning ahead: it asked the AI to first sketch a short list of steps (which columns to group by, which numbers to calculate, whether the answer needs to be ranked *within* each group rather than overall), *before* writing any SQL — and then double-checked the generated SQL actually followed that plan before showing it to you. If that check found something missing, you'll see it happen automatically as an extra entry in the Retry timeline (labeled `plan_not_satisfied`) rather than as a separate error — it's the same self-correction loop described above, just with an extra, more targeted check. An ordinary question (e.g. "how many customers are there?") never shows this expander at all — it's only used when it's likely to help, so simple questions aren't slowed down by it. This same "looks harder than usual" judgment is also what widens the retry limit mentioned above — a hard question quietly gets a few extra attempts rather than being held to the same fixed cap as every other question. Your administrator can turn query planning off entirely (`ENABLE_QUERY_PLANNING=false` in `.env`) if needed; ask them if you're not sure whether it's on.

15. Follow-up questions

You can ask a question that refers back to your previous one — e.g. ask *"What were total sales in 2012?"* and then *"What about 2013 instead?"* The app recognizes this as a follow-up (using your last few successful questions as context) and shows a small caption: *"Following up on: 'What were total sales in 2012?'"* If your question is too ambiguous to tell whether it's a follow-up or a new question, the app will ask you to clarify instead of guessing.

16. Multi-source knowledge: documents, policies, and web search

(Only relevant if your administrator turned this on — skip to §17 otherwise. See `docs/MULTI_SOURCE_GUIDE.md` *for the full setup/configuration reference; this section covers what you'll actually see and do as a user.)*

When enabled, this app can answer more than just database questions — it can also draw on uploaded PDF documents, a separate, more sensitive company-policy collection, and live web search, and it picks which of these (including your database) actually apply to each question automatically, the same way it auto-picks a database in §3. Everything described earlier in this guide (SQL safety, retry behavior, Confirm and Run) still applies unchanged whenever your question turns out to be a database question.

Uploading documents and policies. A separate **"Knowledge Sources"** page (a header nav tab, alongside "Chat" and "Media Search") has two tabs:

- **"Documents"** — general reference material. Upload one or more PDF files; each shows a progress bar, then a success/failure summary (how many chunks were indexed, or the specific error). A document becomes askable as soon as its status turns **ready** in the table below — no app restart needed.
- **"Policies"** — company/HR policy documents, with one extra field: a **sensitivity category** (`None`, `Compensation & pay`, `Disciplinary / HR case content`, `Legal / litigation`). Pick a category only for content that's genuinely restricted — see "Sensitive policies" below for what it actually does. Leave it `None` for ordinary policy content (leave policy, dress code, general process docs).

Each ingested file's row shows its status, upload date, chunk count, and sensitivity category, with a **delete** button to remove it (and everything it contributed) at any time.

Asking a multi-source question. Just ask normally — no need to say which source you mean. A purely database question behaves exactly as described everywhere else in this guide. A question about your documents, policies, or the wider web gets answered directly, with an **"Answered using:"** caption and a badge per source that actually contributed (`Database`, `Documents`, `Policy`, `Web`, ...), and the specific file(s)/URL(s) it drew from underneath. A web-search answer always opens with *"According to a live web search:"* and lists the source links — so it's never confused with your own data. A question that genuinely needs two sources at once (e.g. "compare our leave policy with what's in the database") gets a combined answer with each source's contribution shown under its own labeled heading.

Sensitive policies. A policy document tagged with a sensitivity category is **never summarized into an answer, for anyone**, regardless of role — a fixed, hard-blocked gate with no override. If your administrator has also enabled this app's own accounts (sign-in, top right), a second, independent check can additionally restrict a specific document to named roles (e.g. "HR only") — see `docs/AUTHORIZATION.md`. Asking about a restricted policy returns a fixed message pointing you to HR/the policy owner directly, not a partial or hedged answer.

If a multi-source question comes back wrong or incomplete, it's usually because the question was really two unrelated asks mashed into one sentence — try asking each part separately first.

16.1 Choosing which sources to use

By default the app decides which sources a question needs. If you would rather decide yourself, click the **database icon** next to the chat box to open the source picker.

1. Tick the sources you want this question restricted to. 2. Ask your question as normal. 3. Click **Clear selection** to go back to letting the app choose.

Things worth knowing:

- **Leaving everything unticked is the default**, and means "let the app choose". It is not the same as "no sources".
- **The picker can only narrow, never widen.** Your selection is checked on the server against what is configured *and* what your account is permitted to use. Ticking something you are not entitled to does not grant it — this is a convenience control, not a permission control.
- **Sources you cannot use are still listed, greyed out, with a reason**: *not set up* means your administrator has not configured it, *no access* means your account role does not permit it. Those are different problems with different people to ask.
- **If a source you picked is not available**, your question is still answered from whatever is available, and the answer says so rather than failing or silently substituting.
- **An attached file is always used**, whatever the picker says. Attaching a file to a question is itself a choice, and the app will not discard it.

16.2 Watching a question's progress

Long questions — especially ones that touch several sources — now show what the app is currently doing. The "Thinking 12s…" badge gains the current step, for example *"Thinking 12s… · generate sql"*.

This is a progress indicator, not partial output: the answer itself still arrives in one piece when the pipeline finishes. The steps shown are the app's own fixed pipeline stage names. If you see only "Thinking Ns…" without a step, your deployment is either on an older server or behind a proxy that buffers responses; nothing is wrong, and the answer is unaffected.

17. Voice input and spoken answers

(On by default — your administrator can turn it off; skip to §18 if you don't see a microphone button next to the chat box.)

Click the microphone icon instead of typing, and speak your question. As you talk, word-by-word captions appear in the composer so you can see it being recognized in real time — the box visibly "types" what it hears. The moment you stop talking (or you tap the button again, now showing **Stop**, to end recording manually), the recognized text is dropped straight into the composer for you to review and edit, exactly as if you'd typed it — **nothing is submitted automatically.** Press **Send** (or Enter) when you're ready, the same as any typed question. Once the answer comes back, it's read aloud automatically, in addition to appearing as normal text/results. This is a **record-once, review, then send** pattern, not a continuously-listening or auto-submitting assistant.

A voice-originated question goes through the exact same safety checks, retry behavior, and "Confirm and Run" gate as a typed one — nothing about how it's answered changes because it arrived by voice. You can also edit the recognized text before sending, or clear it and type something else entirely.

If your browser doesn't support live captions, you won't see the word-by-word preview — the composer shows a static *"Recording... speak now, then tap stop"* placeholder instead, and tapping the same button (now showing **Stop**) ends recording, since there's no automatic silence detection to fall back on. Everything else works the same. You can always replay a spoken answer afterward from the small audio control shown under that turn, without it being read aloud a second time automatically.

Transcription and the spoken voice are both produced entirely on your own machine — no cloud speech service, no data leaving it for either direction. The one disclosed exception is the **live caption text** shown while you're still talking: it uses your browser's own built-in speech recognition, which in a Chromium-based browser sends your audio to that browser vendor's own cloud service purely to draw the caption on screen. Your actual submitted question always comes from the local transcription, never from that caption. If you'd rather not see the microphone button at all, look for a **"Voice mode"** toggle in **Settings** (§2/§4) — it hides the control for you without needing your administrator to change anything server-side.

18. Generating images and video

(Only relevant if your administrator turned this on — skip to §19 otherwise.)

Ask for something to be created, rather than something to be looked up — e.g. *"generate an image of monthly spend by category"* or *"create a short video of a factory production line."* The app recognizes this as a request to make brand-new media, as opposed to an ordinary "show me the data" question that merely mentions a picture in passing (which still gets answered as a normal table/chart, not an image).

Nothing is generated, and nothing is charged, until you approve it. Because this is the one feature that spends real, metered credit with an outside provider, the app first shows you what it's *about to* create, with a button — **"Generate image"** or **"Generate video"** — and only calls out to actually create it once you click that. This mirrors the SQL pipeline's own "review before it runs" philosophy (§10), applied to a source that costs money instead of one that reads your database.

Once generated, the image or video is shown directly in the chat — never just a link — served through this app itself rather than pointing you at the provider's own site. A question that combines generation with your database or another source (e.g. *"generate an image of our top 5 merchants by dispute count"*) shows each contribution under its own labeled heading, the same pattern §16 describes for other multi-source answers.

Video clips are short by design — typically 5–15 seconds — a real limit of the underlying generation model, not a restriction this app adds on top.

19. Searching your media library

(Only relevant if your administrator turned this on and pointed it at a local folder of images/videos — skip to §20 otherwise.)

This app can search a local, **untagged** collection of photos and videos by describing what's in them — no filenames, folders, or manual tags required. Two ways to use it:

- **Ask in the main chat**, the same way you'd ask about your database — e.g. *"find the photo of the site inspection from last month"* or *"show me the clip where the crane lifts the beam."* The app tells the difference between this ("find something that already exists") and asking it to generate brand-new media (§18); if in doubt, say "find" or "do we have," not "create" or "make." A matching answer shows a short written summary plus a thumbnail grid of what was found.
- **A dedicated "Media Search" page**, in the navigation bar alongside Chat and Knowledge Sources — type a description, optionally narrow it to images only or videos only, and click **Search**. This works independently of the chat, for quickly browsing the library without starting a conversation.

For a video hit, you'll see a representative frame and a timestamp range (e.g. "1:02–1:18") rather than a playable clip — this feature shows you *where* in the video to look, not a full video player. Search understands on-screen text, spoken audio, and (if your administrator enabled it) an automatically generated description of each clip — so a search can match something said in a video, written on a sign in a photo, or just what a scene visually shows.

Adding new files to the library is an administrator action (running a script after dropping files into the configured folder), not something you do from this page — if a file you know exists never turns up in search results, ask your administrator to confirm the library has been re-indexed. Everything about this feature runs on the same machine as the rest of the app — your media is never uploaded to a third party to make it searchable.

20. Conversation history

The persistent left sidebar lists your **conversations** (most recent first), not individual questions — each conversation is the full back- and-forth from one "New chat" onward. Click **"New chat"** to start a fresh one (nothing is saved until you actually send a first message — see `docs/new-chat-flow.md`), click any conversation to reload it, and use the search box at the top of the sidebar to find a past conversation by title or by anything asked or answered inside it (search runs against the server, not just what's currently loaded in your browser). Each conversation's "■" menu offers **Rename** and **Delete** (a soft delete — see below).

Whether history survives a page refresh or a different device depends on whether you're signed in with a local account (an optional, administrator-enabled feature — see `docs/AUTHENTICATION.md`):

- **Signed in with a local account**: every conversation and message is stored permanently on the server — it follows you across browser tabs, devices, and restarts, and only disappears if you explicitly delete it. See `docs/chat-history-architecture.md`.
- **Not signed in** (the default for a single-user/local setup with no accounts configured): history lives only in the browser tab's memory, exactly like the rest of this guide's Part 1 describes elsewhere — it disappears on refresh or restart, and there is nothing to search on the server.

There is no per-question "re-run" action — reloading a conversation shows you what you asked and were told; to ask it again, just type it into the composer as a new message.

21. Understanding error and status messages

What you see	What it means
"I couldn't process that question. Try rephrasing it..."	Your question was declined before any processing — usually because it wasn't recognized as a database question, or looked like an attempt to manipulate the AI's instructions. The message is intentionally general.
"Needs clarification: ..."	The app couldn't tell what you were really asking (often because a follow-up reference was ambiguous). Rephrase with more detail.
"Agent could not produce a working query: ..."	The AI tried and retried but never produced a query that both passed safety checks and ran successfully. The last attempted SQL is shown below the message.
"You're asking questions faster than I can process them — please wait a moment."	You've hit the per-session question rate limit. Wait briefly and try again.
"The system is handling a lot of requests right now — please wait a moment and try again."	A stricter, shared limit (across all activity, not just yours) was hit mid-processing.
"Rejected: ..." (after clicking Confirm and Run)	The SQL currently in the box — likely one you edited — failed the safety check.
"Execution failed: ..." (after clicking Confirm and Run)	The query passed safety checks but the database itself returned an error (e.g. a genuine timeout).

22. Security — what this app does and doesn't protect against

- Every query that runs is read-only by construction — the app cannot issue `INSERT`/`UPDATE`/`DELETE`/`DROP`, etc., no matter what you type or how the AI responds.
- This is **not** a substitute for using a properly restricted database account — see `SECURITY.md`. If the account configured in `.env` has broader access than read-only, that's a real risk this app's own checks cannot fully cover.
- There is **no login or per-user access control** in this app as shipped — anyone who can open the app's URL can use it with whatever database (and, if enabled, document/policy/web) access is configured. Don't expose it beyond a trusted network without adding your own authentication in front of it (see `docs/DEPLOYMENT.md`).
- If multi-source knowledge (§16) is on, live web search sends your question text to a third-party search API — off unless your administrator explicitly turned it on.
- If voice mode (§17, on by default) is on, the live captions shown while you're speaking use your browser's own built-in speech recognition, which in a Chromium-based browser is cloud-backed — the one part of voice mode that isn't fully local. Your actual submitted question always comes from local transcription, never from the caption text.
- If media search (§19) is on, the content it indexes and searches never leaves your machine — but unlike uploaded documents/policies (§16), there's no sensitivity-tagging step for media library content, so treat anything placed in that folder as visible to anyone who can use this app.
- The AI's accuracy is not perfect — see `docs/EVALUATION.md` for real, measured numbers. Always read the generated SQL and the results before trusting them for anything important.

23. Troubleshooting

See `docs/TROUBLESHOOTING.md` for the full technical reference. The most common day-to-day issues:

- **App won't start / shows a database error screen** — check your `.env` database settings; run `python scripts/test_db_connection.py` for specifics.
- **"Chroma index is empty" or questions never find the right tables** — click **"Refresh Schema"** in Settings (§2/§4), or run `python scripts/build_embeddings.py` from a terminal.
- **Every question is very slow** — expected with a local AI model on modest hardware; this trades speed for running entirely offline. See `docs/EVALUATION.md`'s latency numbers for what's typical.
- **A legitimate question keeps getting rejected** — try rephrasing it more plainly and directly; if it still seems wrong, see `SECURITY.md`'s "Reporting a vulnerability" section.
- **A question that should use documents/policies/web comes back as a database answer (or vice versa)** — see §16 and `docs/MULTI_SOURCE_GUIDE.md` for source-routing specifics.
- **No microphone button appears** — voice mode is off, or your browser denied microphone permission; check your browser's site permissions and ask your administrator to confirm `ENABLE_VOICE_MODE` if it's still missing after that.
- **Spoken answers never play, or you see a voice error** — the one-time Piper voice download may not have been run yet on the server; that's an administrator setup step, not something fixable from the browser.
- **"Generate image/video" never appears, or a generation request is answered as if it were a database question** — media generation is off by default; ask your administrator whether `ENABLE_MEDIA_GENERATION` is set, and try rephrasing with an explicit "generate"/"create" verb.
- **The "Media Search" page says the feature is disabled, or a search never finds a file you know is in the library** — media search is off by default and needs a configured library folder; if it's supposed to be on, ask your administrator to confirm `ENABLE_MEDIA_SEARCH` and that the library has been (re-)indexed after the file was added.

24. Architecture overview

Part 1 above describes what you see and do. From here on, this guide shifts to *how it works* — the actual system design behind that behavior.

Not a free-form AI agent — a small, explicit state machine. The question-answering engine (`agent/graph.py`) is built on LangGraph as a fixed graph of named steps ("nodes") and named transitions ("edges") between them — not a open-ended, ReAct-style agent that decides what to do next by its own free-form reasoning. This is a deliberate choice: every possible path through the system, including every retry, is something you can read off the graph definition, not something that only emerges at runtime inside one large, opaque model call. §25 walks through every node.

One engine, one server process. The React dashboard (`frontend/`) and the REST API it calls (`api/main.py`) are, at the network level, the same process — `api/main.py` serves both. Both ultimately call the exact same `agent.graph.run_agent()` function — neither contains any question- answering logic of its own. Every safety guarantee and behavior described in Part 1 (validation, retries, row caps, rate limits) applies identically regardless of whether a question came through the dashboard or a direct API call.

Fully local by default, with named exceptions. The language model runs locally via Ollama — no API key, no network call, no data leaving the machine for the core question-answering path. The deliberate, disclosed exceptions: live web search (§16/§31), off by default, sends your question text to a third-party search API; voice mode's live captions (§17, on by default) use your browser's own cloud-backed speech recognition for the on-screen caption only, never for the submitted transcript; and media generation (§18, off by default) calls an external provider to actually create an image/video. Media search (§19, off by default) and everything else stay fully local.

Tech stack at a glance:

Layer	Technology
LLM runtime	Ollama, running a local model (default `llama3.1:8b`, swappable)
Orchestration	LangGraph — an explicit state machine, not a black-box agent
Schema retrieval	ChromaDB, a local vector store — see §26
Database connectivity	SQLAlchemy, config-driven (PostgreSQL, MySQL, SQL Server, or Oracle)
SQL validation	sqlglot — AST-based parsing and allowlist checking, see §27
Document/policy storage	SQL Server 2025+/Azure SQL native `VECTOR` columns, see §31
Web search	Pluggable provider, Tavily implemented today
Voice (speech-to-text/text-to-speech)	`faster-whisper` + Piper, both fully local — see §31
Media generation	IMA Studio (image/video), optional, off by default — see §31

Layer	Technology
Media search	Local CLIP embeddings + ChromaDB, `PySceneDetect`/Tesseract/an Ollama vision model for video, optional, off by default — see §31
UI	React + Vite + Tailwind (`frontend/`), served by the API process
API	FastAPI, a thin wrapper with no logic of its own

25. The question-answering pipeline, step by step

Every question passes through the same sequence of nodes. Most run in a fixed order every time; a few only do real work for certain questions (noted below), and a failure at several points can route back to an earlier node rather than simply failing — see §28 for the retry logic. (This is the SQL pipeline specifically; when voice mode, media generation, or media search are involved, an outer router sits in front of it — see §31.)

Node	What it does
`sanitize_input`	The true entry point. Length cap, Unicode normalization (closing a homoglyph-substitution gap plain normalization leaves open), and a regex pre-filter for common prompt-injection phrasings — before anything else touches the question.
`classify_followup`	A cheap heuristic (no model call) deciding whether the question is standalone, a follow-up to your last exchange, or too ambiguous to tell — see §15.
`retrieve_schema`	Embeds the question and retrieves the most relevant tables from ChromaDB (§26). Also where multi-database auto-routing happens, on a database's first pass (§30).
`plan_query`	Only for a question judged non-trivial (§28/§29) — an up-front LLM call sketching the steps the SQL needs to implement, before any SQL is written.
`generate_sql`	Calls the local model with the schema context, the plan (if any), and — on a retry — the previous attempt's error, to produce candidate SQL.
`review_sql`	Only when a plan exists — a second LLM call checking whether the generated SQL actually implements it (§29).
`validate_sql`	Parses the candidate SQL and checks it against a read-only allowlist (§27) — nothing reaches the database without passing this, no exceptions.
`estimate_cost`	A non-executing plan-cost estimate; a query that looks extremely expensive is never run at all.
`execute_sql`	Runs the validated SQL against a read-only connection, with a row cap and timeout.
`generate_insight`	Only after a successful execution — an optional, fact-checked plain-English summary sentence (§12).

In plain sequence: `sanitize_input` → `classify_followup` → `retrieve_schema` → `plan_query` → `generate_sql` → `review_sql` → `validate_sql` → `estimate_cost` → `execute_sql` → `generate_insight`, with `review_sql`/`validate_sql`/`estimate_cost`/`execute_sql` each able to route back to `generate_sql` on a retryable failure (and `execute_sql` able to route all the way back to `retrieve_schema` if the failure suggests the wrong tables were retrieved in the first place).

One state object, `AgentState`, is threaded through every node — each node reads and writes only the fields it's responsible for, and the result of one node becomes the input to the next. This is what makes each

node independently understandable (and independently testable): you can reason about `validate_sql`'s behavior entirely in terms of "given this SQL text, what does it accept or reject," without needing to know anything about how `generate_sql` produced that text.

26. Schema-aware retrieval, technically

Live introspection, never a hardcoded schema. On startup (and on every "Refresh Schema" click), the app reads your database's real structure directly via SQLAlchemy's schema inspector — tables, columns, types, foreign keys — and renders each table as compact, `CREATE TABLE`-style DDL text. Nothing about your schema is ever hardcoded or sampled from a demo database.

Embedding and retrieval. Each table's DDL is embedded (via a local sentence-transformers model — no network call) into ChromaDB, one chunk per table. At question time, a top-k similarity search retrieves the tables that look most relevant to the question's wording — this is what keeps a database with hundreds of tables from overwhelming the model with irrelevant structure.

Value sampling — fixing "the column name lied to me." A column like `ProductLine` looks, from its name alone, like it could hold almost anything; it might actually hold short codes (`M`/`R`/`S`/`T`) with no resemblance to a business term like "Bikes." For string columns under a cardinality cap (20 distinct values or fewer, and never a key column — what keeps this privacy-safe by construction rather than by a column-name denylist), the app embeds the column's *real* sample values directly into its DDL, so the model can match a question's wording against what a column actually contains, not just what it's named.

Filling structural gaps: FK-adjacency bridging. Pure text-similarity retrieval has a blind spot: a fact table (mostly numeric columns) often scores weakly against a question like "which region had the highest sales," even though it's the one table that actually connects "region" to "sales." The app fixes this by treating retrieved tables as a graph and automatically adding any intermediate tables needed to structurally connect them via real foreign keys — not just tables that scored well on wording alone.

Caching. Re-embedding a database's schema is skipped whenever its structure hasn't changed since the last build (checked via a hash of the introspected structure, not the sampled values) — so clicking "Refresh Schema" costs nothing when nothing actually changed.

27. SQL safety architecture

An allowlist on parsed structure, not a blocklist on keywords. Every candidate query — AI-generated or hand-edited — is parsed into a real syntax tree (via `sqlglot`) and checked against an allowlist of the *parsed statement type*: only a single `SELECT`/`UNION`/`EXCEPT`/ `INTERSECT` statement is ever accepted. This is a meaningfully stronger guarantee than a keyword blocklist, which can always be bypassed by a syntax variant it didn't anticipate (unusual casing, comments, string tricks) — there is no way to construct an `INSERT`/`DROP`/etc. statement that parses into the accepted shape.

The whole tree is checked, not just the top. A write or schema- altering operation embedded anywhere in the parsed tree is rejected, even if the outermost statement looks like an ordinary `SELECT` — this closes a real bypass class some engines allow: a data-modifying common-table- expression (`WITH x AS (DELETE FROM t RETURNING *) SELECT * FROM x`) has an entirely ordinary-looking `SELECT` at its root even though it deletes real rows.

A small denylist for known-dangerous functions. Separately, a handful of functions callable from inside an otherwise-ordinary `SELECT` are explicitly blocked (e.g. `pg_sleep`, `xp_cmdshell`, `OPENROWSET`, `UTL_HTTP.REQUEST`) — capable of denial-of-service, file access, or server-side request forgery. Documented as a denylist, not exhaustive, since a list of dangerous functions can only ever be as complete as what's named in it.

Nested-aggregate detection. A more recent addition: SQL like `AVG(CASE WHEN ... THEN SUM(x) ELSE 0 END)` — one aggregate function called inside another's arguments — is rejected by every supported database engine, and was found to be a real, reproducible failure mode (the model reaching for this shape when asked for something like "average year-over-year growth," then exhausting its entire retry budget without ever getting a hint specific enough to escape it). This shape is now caught statically, before the query ever reaches the database, with a rewrite hint pointing at the correct pattern (a pre-aggregating step first, or a window function).

Two outcomes, treated very differently. A rejected query is either an ordinary, retryable mistake (fed back to the model as guidance for another attempt) or a genuine safety violation (non-`SELECT`, a stacked query, an embedded write, a dangerous function call) — the latter fails the whole run immediately, with no retry, on purpose: this is a security gate, not a mistake worth coaching the model through.

Beyond validation: execution-time protections. A row cap is enforced two independent ways — a `LIMIT` clause added to the query text itself, and a hard cap at the database-cursor level, so a malformed or mistranslated query that lacks a working `LIMIT` still can't pull an unbounded result set into memory. A query timeout is enforced by force-closing the connection if it's still running past the configured limit. None of this replaces the need for the configured database account to genuinely be read-only — see §33.

28. Self-correction and the adaptive retry budget

When a query fails — at review, validation, cost-estimation, or execution — the graph doesn't just give up. It routes back to `generate_sql` with the actual error appended to what the model sees next, so each retry has concrete guidance rather than repeating a blind guess. One category of execution failure ("missing reference" — a table/column that doesn't exist) routes all the way back to `retrieve_schema` instead, since the wrong tables may have been retrieved in the first place, not just badly-written SQL.

Not everything is retried. A genuine safety violation or a real query timeout ends the run immediately, since another attempt at the same thing is unlikely to help and would just spend more time/resources on a problem a retry can't fix.

The retry budget adapts to the question. Rather than one fixed number of attempts for every question, a lightweight, LLM-free heuristic scans the question's own wording for signals statistically correlated with needing more self-correction: asking for a result ranked *within* each group rather than overall ("top 3 products *per region*"), a period-over- period comparison ("year-over-year growth"), running/cumulative calculations, or several distinct metrics requested in one question. A question matching one or more of these signals gets extra attempts, up to a configured cap; an ordinary question gets the plain default. This is the same judgment that decides whether query planning (§29) engages at all.

29. Agentic query planning and plan-conformance review

Two extra steps in the pipeline (`plan_query` and `review_sql`, see §25) target the same class of question §28 identifies as non-trivial — a "decompose, then check the decomposition was followed" pattern that complements, rather than replaces, the error-driven retry loop above.

Planning, before any SQL is written. For a question that matches a complexity signal, one LLM call sketches a short, ordered plan: which columns to group by, which numbers to compute, and — critically — whether the result needs per-group ranking (which requires a window function, not a plain row limit combined with grouping) or a period-over-period comparison (which requires comparing pre-aggregated values across periods, never one aggregate nested inside another — see §27). This plan is then included in every subsequent SQL-generation attempt for that question.

Checking the SQL actually followed the plan. After SQL is generated, a second LLM call checks it against that same plan and returns a pass/fail judgment. A failure is treated exactly like any other retryable mistake — the specific critique becomes guidance for the next attempt, sharing the same bounded retry budget as §28, not a second, separate, open-ended loop.

Zero cost for the common case, and fails open. An ordinary question that matches no complexity signal never triggers either call — no added latency, no added model usage, identical behavior to a build without this feature. And if the underlying call to the model fails for any reason (the model server is unreachable, or its response can't be parsed), both steps fail open — proceed as though there were no plan, or as though review passed — since this feature exists to *improve* accuracy, never to become a new reason a question can't be answered. An administrator can also disable it globally regardless of question complexity (`ENABLE_QUERY_PLANNING=false`).

30. Multi-database auto-routing, technically

Each configured database gets its own, separate ChromaDB collection — schemas are never mixed into one shared index. This matters for two reasons: the FK-adjacency bridging in §26 only makes sense within one database's own foreign-key graph, and a shared collection would risk a table-name collision between two unrelated databases that happen to share a table name.

Routing is a cheap pre-step that runs before per-table retrieval: with only one database configured, it's skipped entirely — no extra query, no latency change, identical to a single-database setup. With more than one, each database's own collection is asked for its single best-matching table, and whichever database wins that comparison is used for the question's full retrieval and every downstream step (SQL dialect, connection, engine).

A retry that re-enters schema retrieval (the "missing reference" case in §28) reuses the already-selected database rather than re-running this comparison — a retry must keep targeting the same database the failed attempt already generated SQL against, not silently jump to a different one mid-question.

31. Multi-source orchestration architecture

When the multi-source router is off (the default), the app *is* the SQL pipeline described in §24–§30 — the orchestrator graph below is never even constructed, so nothing in this section changes anything about a SQL-only setup.

When it's on, a router sits in front of the SQL pipeline: it determines which sources are actually available (a source needs both its feature flag *and* its required configuration present — an enabled flag with nothing configured behind it doesn't count as available), then, for a question with two or more available sources, makes one classification call to decide which apply. That call falls back to *every* available source — never zero — if its response can't be confidently parsed, since silently dropping a question is worse than one or two extra source calls. With zero or one source available, this is a free, instant decision (no model call at all). Media generation (§18) and media search (§19) are two more sources this same router can pick, alongside documents/policy/web — the router is what disambiguates "generate a picture of X" (§18) from "find the existing picture of X" (§19) when both are configured.

Parallel fan-out, not sequential. A question needing more than one source (e.g. "compare our leave policy with what's in the database") is routed to every relevant source in parallel within the same processing step, not one after another — each source's own safety boundary (the SQL validator, the policy sensitivity gate, the "web content is untrusted" framing) stays independently testable rather than being folded into one model's implicit judgment about what to do next.

Document/policy storage. Uploaded PDF chunks (both general documents and company policies) are stored using SQL Server 2025+/Azure SQL's native `VECTOR` column type, on a connection dedicated to that storage — deliberately separate from any configured business database, since chunk/embedding storage isn't business data. A policy chunk tagged with a sensitivity category (compensation, disciplinary, or legal) is checked *before* the model is ever asked to summarize it, not filtered after the fact — a hard, tested gate rather than a prompt instruction the model could ignore, because this application has no per-user authorization system capable of deciding who's allowed to see it.

Web search. A pluggable provider interface (Tavily implemented today) means swapping providers is a configuration change, not a code change. Every result is explicitly framed as external, live, untrusted data in the generation prompt — the same "data, not instructions" principle applied to database content and uploaded documents — and the final answer text always identifies itself as coming from a live web search, never presented as if it came from the company's own systems.

Media generation. The one source that spends real, metered money — `Settings.require_generation_approval` (default `true`) means a proposal (`status="pending_approval"`) is created but nothing is actually generated or charged until a human explicitly confirms via `POST /generate/confirm` or the dashboard's "Generate" button (§18), mirroring the SQL pipeline's own "Confirm and Run" gate applied to a source that costs money instead of one that reads your database. A downloaded asset's bytes are served through this app's own `GET /media/{media_id}` route, never the provider's raw URL — the router never even sees a video-vs-image choice made by the caller; `infer_media_kind` re-derives it from the question text server-side every time.

Media search. Unlike every other source, this one runs entirely on-device by default: `media/embedding.py`'s local CLIP model embeds both the search query and every indexed image/video

keyframe into one shared vector space (the same model embeds both, which is what makes them comparable at all), stored in two more ChromaDB collections alongside the schema index. A video is pre-processed once, at index time (`scripts/build_media_index.py`), into scene-detected segments, each transcribed (reusing the same local Whisper model voice mode uses), OCR'd, and optionally captioned by a local Ollama vision model — a blank vision-model setting or a captioning failure just means that segment is still searchable via its transcript/OCR text alone (fails open, the same philosophy as the golden-examples/query-planning features). Retrieved captions/transcripts/OCR text are framed as untrusted data in the answer-composition prompt, never as instructions, the same treatment §31's web-search paragraph above already describes for live search results — a sign in a photo or a spoken phrase in audio is exactly as attacker-influenceable as any other externally-sourced text reaching a prompt.

32. Configuration and deployment

Everything that varies between environments — database connections, the model name, timeouts, every feature flag described throughout this guide — lives in `.env` and is loaded once into a single cached settings object at startup. There is no hardcoded connection string, host, or schema anywhere in the codebase; see `docs/CONFIGURATION.md` for every variable and its default.

The app runs either as a plain Python process (`uvicorn api.main:app`, which serves the built dashboard alongside the API) or via the included `Dockerfile`/`docker-compose.yml` — see `docs/DEPLOYMENT.md` for both paths, including the reverse-proxy-authentication pattern recommended for anything reachable beyond a trusted local network (§22/§33).

Every layer logs through structured logging — deliberately never the connection string, password, or raw result rows, only shape (row/column counts, table names) — and the API exposes a `/health` endpoint that checks the database connection(s), Ollama's reachability, and the schema index in one call, useful for container orchestration health checks.

33. Security model summary

A technical summary of what §22 already covers functionally — see `SECURITY.md` for the complete reference this section condenses.

What's actually enforced: read-only-by-construction SQL (the §27 allowlist, checked on every single query regardless of origin), a database account that should itself be genuinely read-only (a second, independent layer — the app's own checks are not a substitute for this), a row cap and query timeout enforced at execution time, per-session and process-wide rate limiting, secret redaction in every log line, and Unicode-normalization plus pattern-based defenses against prompt-injection attempts (the same "untrusted until checked" treatment §27 applies to generated SQL and §31 applies to database content, uploaded documents, media-library content, and web results).

What's explicitly not guaranteed, by design: there is no login or per-user authorization system — anyone who can reach the app's URL can use whatever access is configured, so it's not designed for exposure beyond a trusted network without a real authenticating reverse proxy in front of it. This project has not been through an independent security review or penetration test. Neither is there any per-item sensitivity classification for media library content (§19/§31) the way policy documents have (§16) — if that library could contain something restricted, that's a per-deployment gap to solve before turning media search on, not a control this app provides. All of these are named, documented limitations, not silent gaps — see `SECURITY.md`'s "What is explicitly not guaranteed" section for the complete, honest list.

34. Further technical reading

This guide's Part 2 is a condensed summary. For full depth:

- `docs/ARCHITECTURE.md` — the complete technical walkthrough: every node's full behavior, the complete retry-routing table, and the schema-retrieval and multi-source pipelines in full detail.
- `SECURITY.md` — the complete security reference, including what's explicitly not guaranteed.
- `docs/CONFIGURATION.md` — every `.env` variable and its default.
- `docs/API.md` — the REST API reference.
- `docs/MULTI_SOURCE_GUIDE.md` — how to configure and use each optional source.
- `docs/EVALUATION.md` — real, measured accuracy and latency numbers, not just methodology.
- `docs/DEPLOYMENT.md` — running this in Docker and behind a reverse proxy.
- `CONTRIBUTING.md` — for anyone extending the codebase, including how to add a benchmark case.
- `docs/GOVERNANCE.md`, `docs/COMPLIANCE.md`, `docs/RESPONSIBLE_AI.md`, and `docs/RISK_REGISTER.md` — for a governance- or compliance-focused reader.